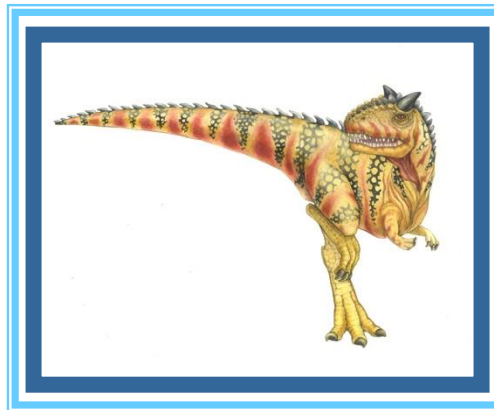


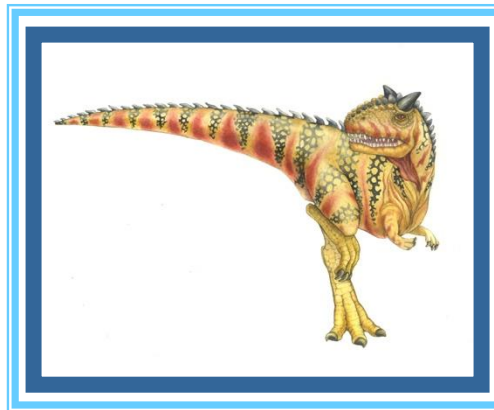
These slides have been modified from the original



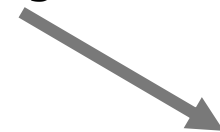
A huge thanks to the
authors of Operating
System Concepts for
providing this resource



These slides have been modified from the original



An indicator on a slide means I've modified something on the slide from the original slides



Modified
slide

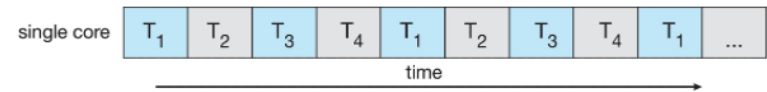
Synchronisation



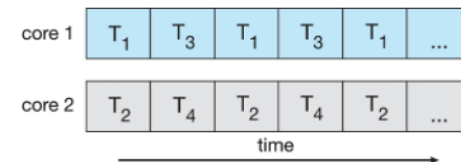
Background

- Processes can execute concurrently/in parallel
 - May be **interrupted** at any time, **partially completing execution**
- Concurrent access to shared data may result in data inconsistency
- So, maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
- We illustrated the with this simple example before.... which lead to a race condition.

- Concurrent execution on single-core system:



- Parallelism on a multi-core system:



- `counter++` could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- `counter--` could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with “count = 5” initially:

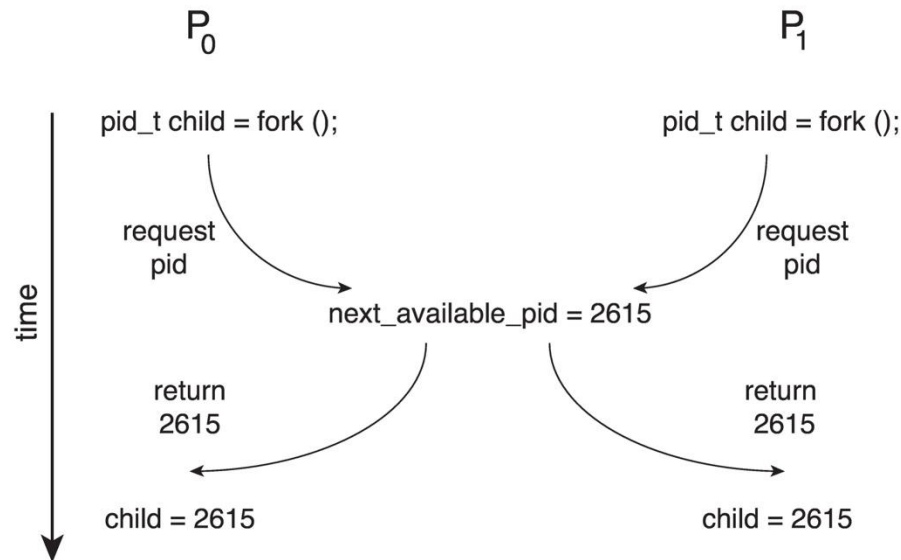
S0: producer execute <code>register1 = counter</code>	{register1 = 5}
S1: producer execute <code>register1 = register1 + 1</code>	{register1 = 6}
S2: consumer execute <code>register2 = counter</code>	{register2 = 5}
S3: consumer execute <code>register2 = register2 - 1</code>	{register2 = 4}
S4: producer execute <code>counter = register1</code>	{counter = 6}
S5: consumer execute <code>counter = register2</code>	{counter = 4}

Modified slide



Race Condition

- Processes P_0 and P_1 are creating child processes using the `fork()` system call
- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)



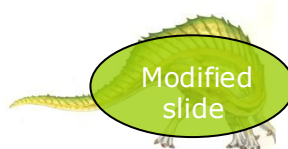
- Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable `next_available_pid` the same pid could be assigned to two different processes!





Critical Section Problem

- Consider system of n processes $\{p_0, p_1, \dots p_{n-1}\}$
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, **writing file**, etc.
 - When one process in critical section, **no other may be in its critical section**
- **Critical section problem** is to design **protocol to solve this**
- Often structured: Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**





Critical Section

- General structure of process P_i

```
while (true) {
```

```
    entry section
```

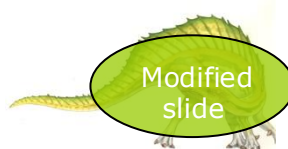
```
    critical section
```



```
    exit section
```

```
    remainder section
```

```
}
```

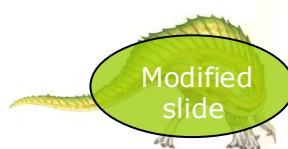




Critical-Section Problem (Cont.)

Requirements for solution to critical-section problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the process that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted

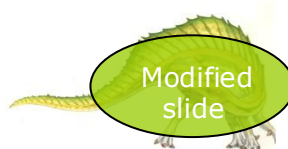




Interrupt-based Solution

- Entry section: **disable interrupts**
- Exit section: **enable interrupts**
- Will this solve the problem?
 - What if the critical section is code that runs for an hour?
 - What if there are two CPUs?
 - ...

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```

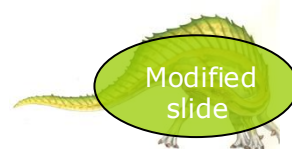




Peterson's Algorithm

- (A software solution)
- Two process solution
- Assume that the **load** and **store** machine-language instructions are **atomic**; that is, cannot be interrupted
- The **two processes** share **two variables**:
 - `int turn;`
 - `boolean flag[2]`
- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section.
 - `flag[i] = true` implies that process P_i is ready!

```
while (true) {  
    entry section  
        critical section  
    exit section  
        remainder section  
}
```





Peterson's Algorithm

P0

```
while (true){
```

```
    flag[0] = true;  
    turn = 1;  
    while (flag[1] && turn == 1);
```

```
    /* critical section here */
```

```
    flag[0] = false;
```

```
    /* remainder section here */
```

```
}
```

P1

```
while (true){
```

```
    flag[1] = true;  
    turn = 0;  
    while (flag[0] && turn == 0);
```

```
    /* critical section here */
```

```
    flag[1] = false;
```

```
    /* remainder section here */
```

```
}
```





Correctness of Peterson's Solution

- Provable that the three CS requirement are met:
 1. Mutual exclusion is preserved
 - P_i enters CS only if:
either **flag[j] = false** or **turn = i**
 2. Progress requirement is satisfied
 3. Bounded-waiting requirement is met

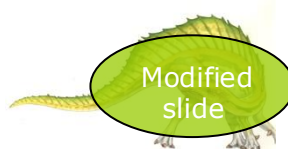
If one process doesn't want in ($\text{flag}[j] = \text{false}$), the other proceeds immediately.
Once P_i sets $\text{flag}[i] = \text{true}$, P_j can enter at most once before P_i enters.





Peterson's Solution and Modern Architecture

- Although useful for demonstrating an algorithm, Peterson's Solution is not guaranteed to work on modern architecture memory model semantics.
 - **Why?** To improve performance, processors and/or compilers may reorder operations that have no dependencies (**out-of-order** execution), and there's cache coherence visibility delay.
- Understanding why it will not work is useful for better understanding race conditions.
- Compilers and CPUs may reorder loads/stores or delay their visibility to other cores (relaxed memory models).





Modern Architecture Example

- Two threads share the data:

```
boolean flag = false;  
int x = 0;
```

- Thread 1 performs

```
while (!flag);  
print x
```

- Thread 2 performs

```
x = 100;  
flag = true
```

- What is the expected output?
100





Modern Architecture Example (Cont.)

- However, since the variables `flag` and `x` are independent of each other (no data dependency between the writes), the instructions:

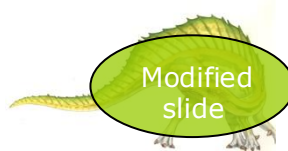
```
flag = true;  
x = 100;
```



The compiler/CPU
may reorder the
writes as ..

for Thread 2 may be reordered

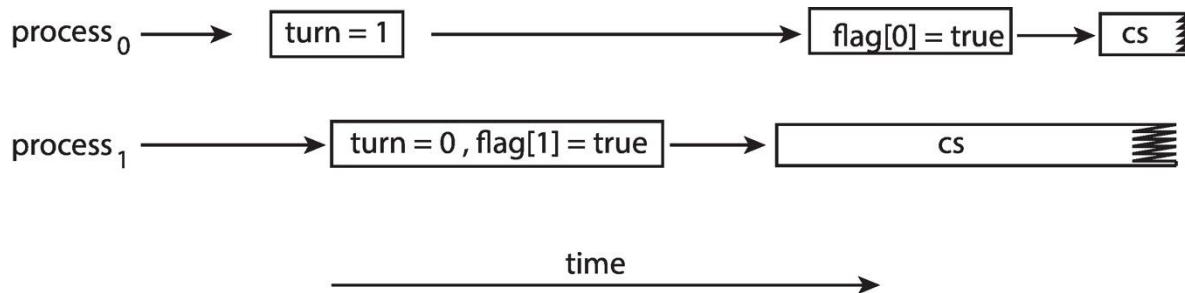
- If this occurs, the output may be 0!





Peterson's Solution Revisited

- The effects of instruction reordering in Peterson's Solution



- This allows both processes to be in their critical section at the same time!
- To ensure that Peterson's solution will work correctly on modern computer architecture we must use **Memory Barrier**.





Algorithm for Process P_i

P0

```
while (true){
```

```
    flag[0] = true;  
    turn = 1;  
    while (flag[1] && turn == 1);
```

```
    /* critical section here */
```

```
    flag[0] = false;
```

```
    /* remainder section here */
```

```
}
```

P1

```
while (true){
```

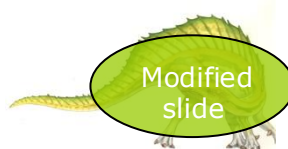
```
    flag[1] = true;  
    turn = 0;  
    while (flag[0] && turn == 0);
```

```
    /* critical section here */
```

```
    flag[1] = false;
```

```
    /* remainder section here */
```

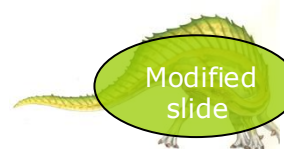
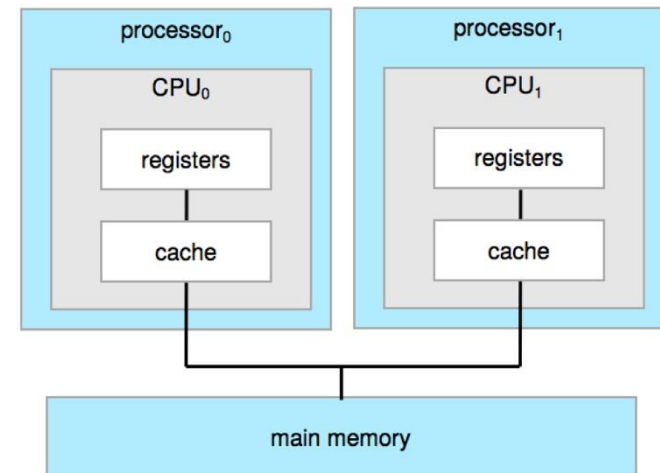
```
}
```





Memory Barrier

- **Memory models** are the memory guarantees a computer architecture makes to application programs.
- A **memory barrier** enforces ordering constraints on memory operations and prevents certain loads/stores from being reordered across it.





Memory Barrier Instructions

- When a **full memory fence** instruction is performed, the system ensures that **all loads and stores are completed before any subsequent load or store operations are performed**.
- Therefore, even if instructions were reordered, the memory barrier ensures that the store operations are completed and visible to other processors before future load or store operations are performed.

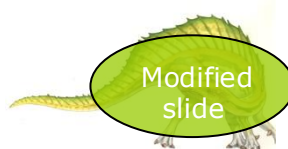
In c we can use functions like...

```
__sync_synchronize();
```

```
__atomic_thread_fence(__ATOMIC_SEQ_CST);
```

... no memory operand will be moved across the operation, either forward or backward.

<https://gcc.gnu.org/onlinedocs/gcc-4.6.2/gcc/Atomic-Builtins.html>



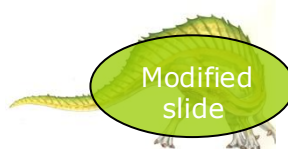


Memory Barrier Example

- Returning to the example ...
- We could add a memory barrier to the following instructions to ensure Thread 1 outputs 100:
- Thread 1 now performs

```
while (!flag);  
memory_barrier();  
print x;
```
- Thread 2 now performs

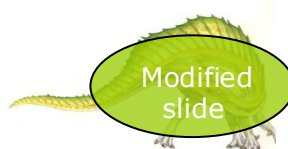
```
x = 100;  
memory_barrier();  
flag = true;
```
- If Thread 1 observes `flag==True`, then it is guaranteed to observe all writes that happened-before e.g. `x`.
- For Thread 2 we ensure that the assignment to `x` occurs before the assignment `flag`.





Synchronization Hardware

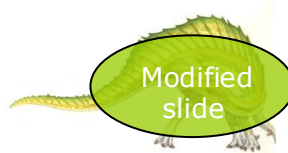
- Many systems provide hardware support for implementing critical section code.
- On a single-core system, disabling interrupts prevents preemption and therefore ensures mutual exclusion.
 - Currently running thread would execute **without preemption**
 - Not sufficient on multiprocessor systems, since other CPUs continue executing concurrently.
 - Also poor for performance and responsiveness.
- For multiprocessor systems, we need hardware instructions to solve the critical section problem.





Hardware Instructions

- **Special hardware instructions** that allow us to either *test-and-modify* the content of a word, or to *swap* the contents of two words **atomically** (operation appears indivisible to other processors)
 - **Test-and-Set** instruction
 - **Compare-and-Swap** instruction





The test_and_set Instruction

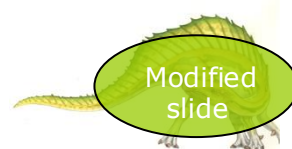
■ Definition

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = true;
    return rv;
}
```

Note: this is C code to show the operation of the test_and_set instruction, this C code is not executed.

■ Properties

- **Executed atomically by hardware** as a single read–modify–write operation
- Returns the original value of passed parameter
- Set the new value of passed parameter to **true**





Solution Using test_and_set()

- Shared boolean variable `lock`, initialized to `false`
- Solution:

```
do {  
    while (test_and_set(&lock));  
    /* do nothing */  
  
    /* critical section here */  
  
    lock = false;  
    /* remainder section here */  
} while (true);
```

```
boolean test_and_set (boolean *target)  
{  
    boolean rv = *target;  
    *target = true;  
    return rv;  
}
```

Modified
slide



The compare_and_swap Instruction

■ Definition

```
int compare_and_swap(int *value, int expected, int new_value)
{
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

Note: this is C code to show the operation of the compare_and_swap instruction, this C code is not executed.

■ Properties

- **Executed atomically by hardware** as a single read–modify–write operation
- Returns the original value of passed parameter **value**
- Set the variable **value** the value of the passed parameter **new_value** but only if ***value == expected** is true. That is, the swap takes place only under this condition.





Solution using compare_and_swap

- Shared integer `lock` initialized to 0;
- Solution:

```
while (true){  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = 0;  
  
    /* remainder section */  
}
```





Atomic Variables

- Typically, instructions such as `test_and_set` and `compare_and_swap` are used as building blocks for other synchronization tools.
- One tool is an **atomic variable** that provides *atomic* updates on basic data types such as integers and booleans.
- For example:
 - Let **sequence** be an atomic variable
 - Let **increment()** be operation on the atomic variable **sequence**
 - The Command:

increment(&sequence) ;

ensures **sequence** is incremented without interruption.





Atomic Variables

- The `increment()` function can be implemented as follows:

```
void increment(atomic_int *v)
{
    int temp;
    do {
        temp = *v;
    }
    while (temp != (compare_and_swap(v, temp, temp+1)) );
}
```

```
int compare_and_swap(int *value, int expected, int new_value)
{
    int t = *value;
    if (*value == expected)
        *value = new_value;
    return t;
}
```



```
#include <stdio.h>
#include <threads.h>
#include <stdatomic.h> ←
atomic_int acnt=0;
int cnt=0;

int f(void* thr_data)
{
    for(int n = 0; n < 1000; ++n) {
        ++cnt;
        ++acnt;
    }
    return 0;
}

int main(void)
{
    thrd_t thr[10];
    for(int n = 0; n < 10; ++n)
        thrd_create(&thr[n], f, NULL);
    for(int n = 0; n < 10; ++n)
        thrd_join(thr[n], NULL);

    printf("The atomic counter is %u\n", acnt);
    printf("The non-atomic counter is %u\n", cnt);
}
```

From: <https://en.cppreference.com/w/c/language/atomic>

To note... Hardware-based solutions to the critical section problem like `test_and_set` and `compare_and_swap` are typically inaccessible to programmers... You don't use them directly!

Programmers are provided with functionality by the operating system to solve the critical section problem.

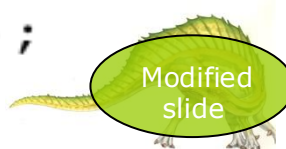


Mutex Locks

Simplest of sync tools

- Previous solutions are complicated and generally inaccessible to application programmers
- OS designers build software mechanisms to solve critical section problem
 - Mutex is synchronization primitive that provides mutual exclusion
- There are also semaphores, etc
 - Protect a critical section by
 - ▶ First **acquire()** a lock (**and blocks until it can**)
 - ▶ Then **release()** the lock
 - Where calls to **acquire()** and **release()** are **atomic**
- But our previous solutions require **busy waiting**
 - This lock therefore called a **spinlock**

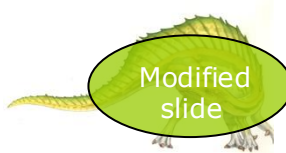
```
while (test_and_set(&lock));
```





Solution to CS Problem Using Mutex Locks

```
while (true) {  
    acquire lock  
  
    critical section  
  
    release lock  
  
    remainder section  
}
```





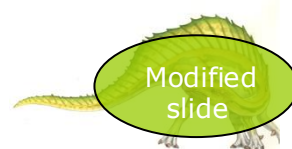
Semaphore

- Semaphore: A general synchronization primitive used to coordinate threads or processes.
- Semaphore **S** , integer variable
- Can only be accessed via two indivisible (**atomic**) operations
 - **wait()** and **signal()**
 - ▶ Originally called **P()** and **V()**
- Definition of the **wait()** operation

```
wait(S) {  
    while (S == 0)  
        ; // busy wait  
    S--;  
}
```

Still have some busy waiting... In reality it's implemented differently, covered in a bit

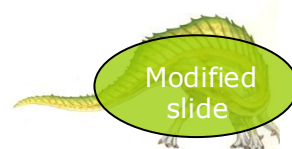
- Definition of the **signal()** operation
- ```
signal(S) {
 S++;
}
```





# Semaphore (Cont.)

- **Binary semaphore** – integer value can range only between 0 and 1
  - Can be used as a **mutex lock**
- **Counting semaphore** – integer value can range over an unrestricted domain (resource counting)
- binary semaphore is a special case of a counting semaphore **S** ... as a binary semaphore  $S=1$  so values can be 0 or 1
- With semaphores we can solve various synchronization problems





# Semaphore Usage Example

- Solution to the CS Problem
  - Create a semaphore “**mutex**” initialized to 1

```
wait(mutex) ;
CS
signal(mutex) ;
```
- Consider  $P_1$  and  $P_2$  that with two statements  $S_1$  and  $S_2$  and the requirement that  $S_1$  to happen before  $S_2$ 
  - Create a semaphore “**synch**” initialized to 0

```
P1 :
 S1 ;
 signal(synch) ;
P2 :
 wait(synch) ;
 S2 ;
```

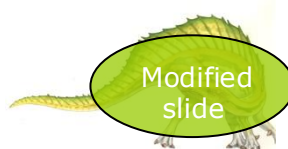




# Semaphore Implementation

- Guarantee that no two processes can execute the **wait()** or **signal()** on the same semaphore at the same time
- We use the OS implementation .... as .... there's a problem where the **wait** and **signal** code and their critical section
- What about **busy waiting** in critical section implementation?
  - Maybe little busy waiting if critical section rarely occupied
- but applications may spend lots of time in critical sections and therefore **this is not a good solution**

```
wait(S) {
 while (S <= 0)
 ; // busy wait
 S--;
}
```

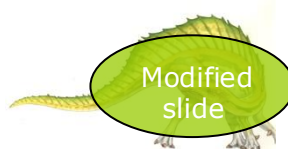




# Semaphore Implementation with no Busy waiting

---

- The solution to this?
- With each semaphore there is an associated **waiting queue**
- Each **waiting queue** has two data items:
  - Value (of type integer)
  - Pointer to next record in the list
- Two operations:
  - **a way to Block** – place the process invoking the operation on the appropriate waiting queue
  - **a way to Wakeup** – remove one of processes in the waiting queue and place it in the ready queue





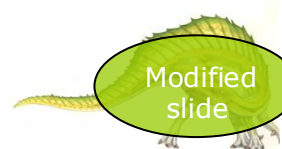
# No busy waiting

```
wait(S) {
 while (S <= 0)
 ; // busy wait
 S--;
}
```

```
struct semaphore {
 int value;
 queue L; // list of processes
}
```

```
wait (S) {
 if (s.value > 0)
 s.value = s.value - 1; else
{
 add this process to s.L;
 block;
 }
```

```
signal (S) {
 if (S.L != EMPTY) {
 remove a process from S.L;
 wakeup(P);
 }
 else
 s.value = s.value + 1;
}
```





# No busy waiting

```
wait(S) {
 while (S <= 0)
 ; // busy wait
 S--;
}
```

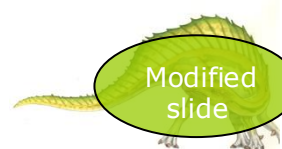
```
struct semaphore {
 int value;
 queue L; // list of processes
}
```

```
wait (S) {
 acquire_internal_lock()
 if (s.value > 0)
 s.value = s.value - 1;
 else
 {
 add this process to s.L;
 release_internal_lock()
 block;
 return;
 }
 release_internal_lock()
}
```

**wait()** and **signal()** must be atomic, so they still have a lock, but in reality a thread now spends very little time in the CS of **wait()** and **signal()**

```
signal (S) {
 acquire_internal_lock()
 if (S.L != EMPTY) {
 remove a process from S.L;
 wakeup(P);
 }
 else
 s.value = s.value + 1;
 release_internal_lock()
}
```

Semaphores eliminate busy waiting for user threads, but internally the kernel still uses a very small spinlock to protect the semaphore structure.





# Problems with Semaphores

---

- Incorrect use of semaphore operations:
  - `signal(mutex) .... wait(mutex)`
  - `wait(mutex) ... wait(mutex)`
  - Omitting of `wait(mutex)` and/or `signal(mutex)`
- These – and others – are examples of what can occur when semaphores and other synchronization tools are used incorrectly.



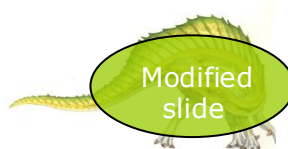




# Liveness

---

- **Liveness** refers to a set of properties that a system must satisfy to ensure processes make progress.
- Processes should not have to **wait indefinitely** while trying to acquire a synchronization tool such as a mutex lock or semaphore.
- Waiting indefinitely violates the progress and bounded-waiting criteria discussed earlier.
- Indefinite waiting is an example of a liveness failure.





# Liveness

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let  $S$  and  $Q$  be two semaphores initialized to 1

$P_0$   
`wait(S) ;`  
`wait(Q) ;`  
`...`  
`signal(S) ;`  
`signal(Q) ;`

$P_1$   
`wait(Q) ;`  
`wait(S) ;`  
`...`  
`signal(Q) ;`  
`signal(S) ;`

- Consider if  $P_0$  executes `wait(S)` and  $P_1$  `wait(Q)`. When  $P_0$  executes `wait(Q)`, it must wait until  $P_1$  executes `signal(Q)`
- However,  $P_1$  is waiting until  $P_0$  execute `signal(S)`.
- Since these `signal()` operations will never be executed,  $P_0$  and  $P_1$  are **deadlocked**.



Let's take what we know now about semaphores and mutexes, and see how they can be used in solving various problems of synchronization ...



# Classical Problems of Synchronization

---

- Classical problems used to test newly-proposed synchronization schemes
  - Bounded-Buffer Problem (aka Producer-Consumer problem)
  - Readers and Writers Problem
  - Dining-Philosophers Problem





- Remember... We look at the bounded buffer (producer-consumer problem) before

```
#define BUFFER_SIZE 10
typedef struct {
 . . .
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

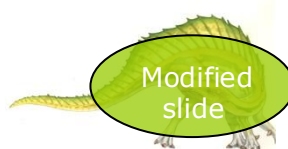
```
item next_produced;

while (true) {
 /* produce an item in next produced */
 while (((in + 1) % BUFFER_SIZE) == out)
 ; /* do nothing */
 buffer[in] = next_produced;
 in = (in + 1) % BUFFER_SIZE;
}
```

```
item next_consumed;

while (true) {
 while (in == out)
 ; /* do nothing */
 next_consumed = buffer[out];
 out = (out + 1) % BUFFER_SIZE;

 /* consume the item in next consumed */
}
```

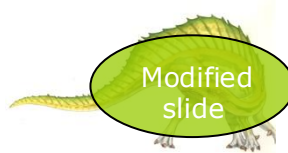




# Bounded-Buffer/Producer Consumer

---

- $n$  buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1 Protects critical section
- Semaphore **full** initialized to the value 0 Goes from 0 ...  $n$
- Semaphore **empty** initialized to the value  $n$  Goes from  $n$  ... 0





# Bounded-Buffer/Producer Consumer

---

- The structure of the producer process

```
while (true) {
 ...
 /* produce an item in next_produced */
 ...
 wait(empty) ;
 wait(mutex) ;
 ...
 /* add next produced to the buffer */
 ...
 signal(mutex) ;
 signal(full) ;
}
```





# Bounded-Buffer/Producer Consumer

- The structure of the consumer process

```
while (true) {
 wait(full);
 wait(mutex);
 ...
 /* remove an item from buffer to next_consumed */
 ...
 signal(mutex);
 signal(empty);
 ...
 /* consume the item in next consumed */
 ...
}
```





# Bounded-Buffer/Producer Consumer

- The structure of the consumer process

```
while (true) {
 wait(full);
 wait(mutex);

 ...
 /* remove an item */

 ...
 signal(mutex);
 signal(empty);

 ...
 /* consume the item */

 ...
}
```

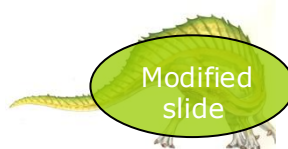
- The structure of the producer process

```
while (true) {
 ...
 /* produce an item */
 ...
 wait(empty);
 wait(mutex);

 ...
 /* add next produced*/

 ...
 signal(mutex);
 signal(full);

}
```



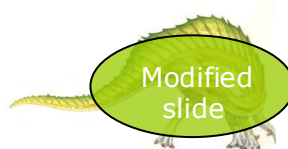


# Semaphore

```
struct semaphore {
 int value;
 queue L; // list of processes
}

wait (S) {
 if (s.value > 0)
 s.value = s.value - 1; else
 {
 add this process to s.L;
 block;
 }

signal (S) {
 if (S.L != EMPTY) {
 remove a process from S.L;
 wakeup(P);
 }
 else
 s.value = s.value + 1;
}
```

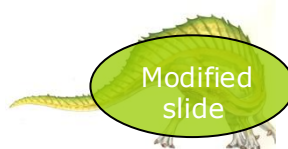




# Readers-Writers Problem

---

- A data set is shared among a number of concurrent processes
  - **Readers** – only read the data set; they do *not* perform any updates (**hence multiple readers at the same time is okay!**)
  - **Writers** – can both read and write (**but only one writer at a time, and no readers when writing**)
- Problem – allow multiple readers to read at the same time
  - Though... only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities
  - First, second and third

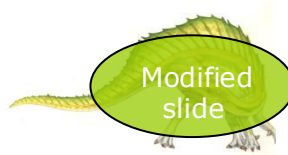




# Readers-Writers Problem (Cont.)

---

- Shared Data
  - Data set
  - Semaphore **rw\_mutex** initialized to 1
  - Semaphore **mutex** initialized to 1 (**used to protect read\_count**)
  - Integer **read\_count** initialized to 0 (**how many are reading – and needs to be protected**)





# Readers-Writers Problem (Cont.)

- The structure of a writer process

```
while (true) {
 wait(rw_mutex);

 ...
 /* writing is performed */
 ...
 signal(rw_mutex);
}
```



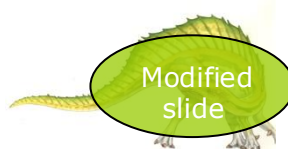


# Readers-Writers Problem (Cont.)

- The structure of a reader process

```
while (true){
 wait(mutex);
 read_count++;
 if (read_count == 1) /* first reader */
 wait(rw_mutex);
 signal(mutex);

 ...
 /* reading is performed */
 ...
 wait(mutex);
 read_count--;
 if (read_count == 0) /* last reader */
 signal(rw_mutex);
 signal(mutex);
}
```





# Readers-Writers Problem Variations

---

- The solution in previous slide can result in a situation where a writer process **never writes**. It is referred to as the “First reader-writer” problem.
- The “Second reader-writer” problem is a variation the first reader-writer problem that state:
  - Once a writer is ready to write, no “newly arrived reader” is allowed to read.
- Both the first and second may result in starvation, leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks

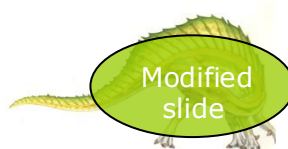




# Readers-Writers Problem

---

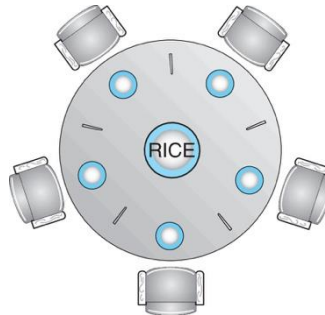
- Several variations of how readers and writers are considered – all involve some form of priorities
  - **First readers-writers problem:** no reader should be kept waiting, unless a writer has **already** obtained permissions (readers-preference)
  - **Second readers-writers problem:** if a writer is ready to write, it writes as soon as possible i.e., takes priority over waiting readers (writers-preference)
  - **Third:** that a reader nor writer shall starve (because 1 & 2 could create starvation)



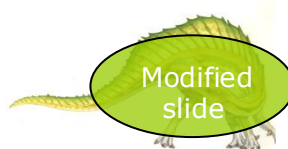


# Dining-Philosophers Problem

- N philosophers' sit at a round table with a bowl of rice in the middle.



- They spend their lives alternating between thinking and eating.
- They do not interact with their neighbors.
- Occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done
- In the case of 5 philosophers, the shared data
  - ▶ Bowl of rice (data set)
  - ▶ Semaphore chopstick [5] initialized to 1





# Dining-Philosophers Problem Algorithm

- Semaphore Solution
- The structure of Philosopher  $i$  :

```
while (true){
 wait (chopstick[i]);
 wait (chopstick[(i + 1) % 5]);

 /* eat for awhile */

 signal (chopstick[i]);
 signal (chopstick[(i + 1) % 5]);

 /* think for awhile */

}
```

- What is the problem with this algorithm?

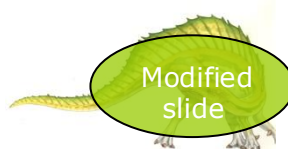




# POSIX Synchronization

---

- POSIX API provides
  - mutex locks
  - semaphores
  - condition variables
  - atomic variables
  
- Widely used on UNIX, Linux, and macOS





# POSIX Mutex Locks

---

- Creating and initializing the lock

```
#include <pthread.h>
```

```
pthread_mutex_t mutex;
```

```
/* create and initialize the mutex lock */
pthread_mutex_init(&mutex, NULL);
```

- Acquiring and releasing the lock

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);
```

```
/* critical section */
```

```
/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```





# POSIX **Unnamed** Semaphores

---

- Creating and initializing the semaphore:

```
#include <semaphore.h>
sem_t sem;

/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```





# POSIX Named Semaphores

- Creating and initializing the semaphore:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

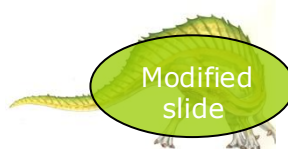
- Another process can access the semaphore by referring to its name **SEM**.
- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```

Named semaphores can be used by unrelated processes, unnamed cannot.





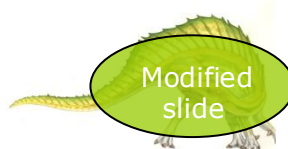
# POSIX Condition Variables

---

- Creating and initializing the condition variable:

```
pthread_mutex_t mutex;
pthread_cond_t cond_var;
```

```
pthread_mutex_init(&mutex, NULL);
pthread_cond_init(&cond_var, NULL);
```





# POSIX Condition Variables

---

- Thread waiting for the condition `a == b` to become true:

```
pthread_mutex_lock(&mutex);
while (a != b)
 pthread_cond_wait(&cond_var, &mutex);
a = 100;
pthread_mutex_unlock(&mutex);
```

- Thread signaling another thread waiting on the condition variable:

```
pthread_mutex_lock(&mutex);
a = b;
pthread_cond_signal(&cond_var);
pthread_mutex_unlock(&mutex);
```





# Linux Synchronization

- Atomic variables

`atomic_t` is the type for atomic integer

- Consider the variables

```
atomic_t counter;
int value;
```

| <i>Atomic Operation</i>                         | <i>Effect</i>                       |
|-------------------------------------------------|-------------------------------------|
| <code>atomic_set(&amp;counter,5);</code>        | <code>counter = 5</code>            |
| <code>atomic_add(10,&amp;counter);</code>       | <code>counter = counter + 10</code> |
| <code>atomic_sub(4,&amp;counter);</code>        | <code>counter = counter - 4</code>  |
| <code>atomic_inc(&amp;counter);</code>          | <code>counter = counter + 1</code>  |
| <code>value = atomic_read(&amp;counter);</code> | <code>value = 12</code>             |





```
#include <stdio.h>
#include <threads.h>
#include <stdatomic.h>
```

```
atomic_int acnt;
int cnt;
```

```
int f(void* thr_data)
{
 for(int n = 0; n < 1000; ++n) {
 ++cnt;
 ++acnt;
 }
 return 0;
}
```

```
int main(void)
{
 thr_t thr[10];
 for(int n = 0; n < 10; ++n)
 thr_create(&thr[n], f, NULL);
 for(int n = 0; n < 10; ++n)
 thr_join(thr[n], NULL);

 printf("The atomic counter is %u\n", acnt);
 printf("The non-atomic counter is %u\n", cnt);
}
```

From: <https://en.cppreference.com/w/c/language/atomic>



# Deadlocks



# Deadlock with Semaphores

---

- Data:
  - A semaphore **s1** initialized to 1
  - A semaphore **s2** initialized to 1
- Two processes **P1** and **P2**
- **P1**:

```
wait(s1)
wait(s2)
```
- **P2**:

```
wait(s2)
wait(s1)
```

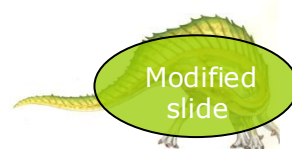




# System Model

---

- System consists of resources
- Resource types  $R_1, R_2, \dots, R_m$ 
  - *CPU cycles, memory space, I/O devices*
- Each resource type  $R_i$  has  $W_i$  instances.
- Each process utilizes a resource as follows:
  - **request**
  - **use**
  - **release**
  - (and **claim** we will see in a moment)





# Resource-Allocation Graph

A set of vertices  $V$  and a set of edges  $E$ .

- $V$  is partitioned into two types:
  - $T = \{T_1, T_2, \dots, T_n\}$ , the set consisting of all the processes in the system
  - $R = \{R_1, R_2, \dots, R_m\}$ , the set consisting of all resource types in the system
- **request edge** – directed edge  $T_i \rightarrow R_j$    $T_i$  requesting  $R_j$
- **assignment edge** – directed edge  $R_j \rightarrow T_i$    $R_j$  allocated to  $T_i$

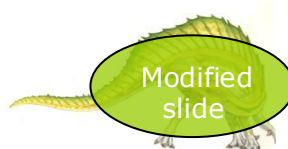




# Deadlock Characterization

Deadlock can arise if **four conditions** hold **simultaneously**.

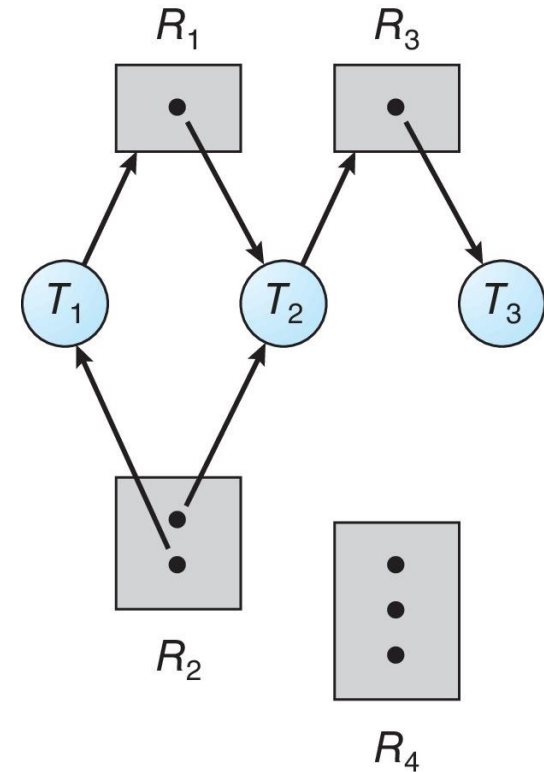
- **Mutual exclusion:** only one process at a time can use a resource (non sharable e.g. writing to a file)
  - Another process must wait for it
- **Hold and wait:** a process **must be holding at least one resource, and** is **waiting** to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task i.e., **resources cannot be preempted**
- **Circular wait:** there exists a set  $\{P_0, P_1, \dots, P_n\}$  of waiting processes such that  $P_0$  is waiting for a resource that is held by  $P_1$ ,  $P_1$  is waiting for a resource that is held by  $P_2$ , ...,  $P_{n-1}$  is waiting for a resource that is held by  $P_n$ , and  $P_n$  is waiting for a resource that is held by  $P_0$ .





# Resource Allocation Graph Example

- One instance of R1
- Two instances of R2
- One instance of R3
- Three instance of R4
- T1 holds one instance of R2 and is waiting for an instance of R1
- T2 holds one instance of R1, one instance of R2, and is waiting for an instance of R3
- T3 is holds one instance of R3

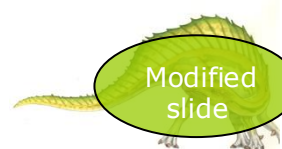
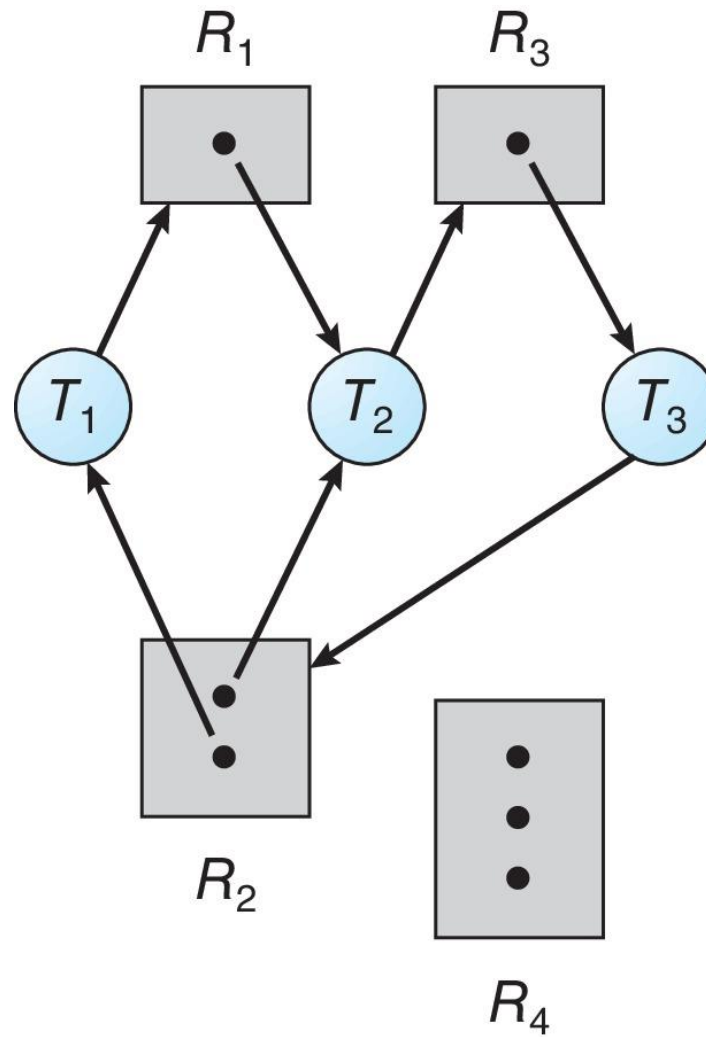


**No deadlock**



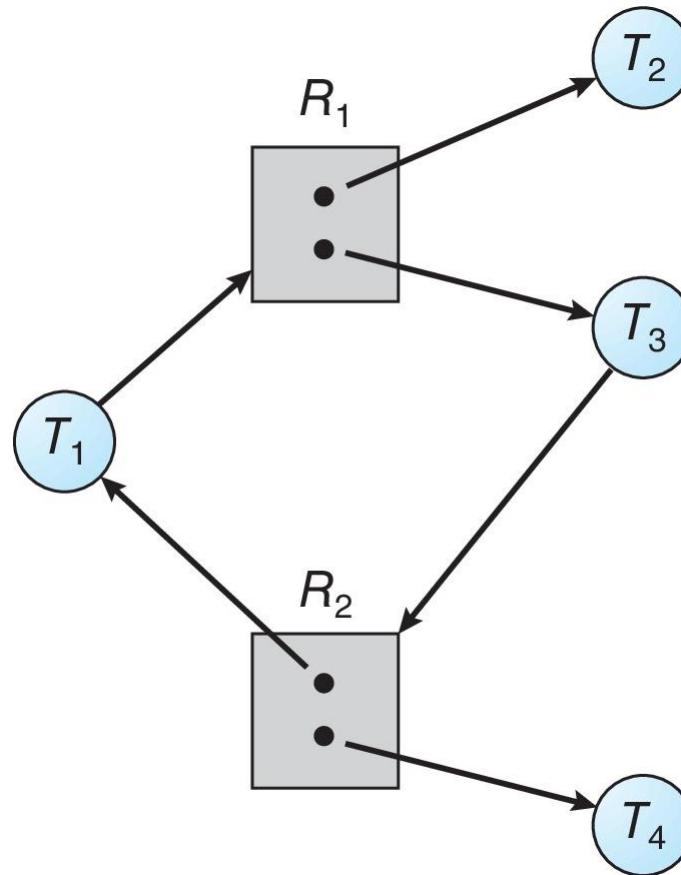


# Resource Allocation Graph with a **Deadlock**

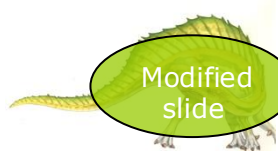




# Graph with a Cycle But **no Deadlock**



**Why?**

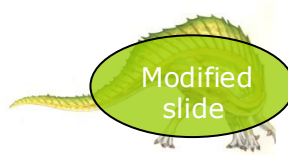




# Basic Facts

---

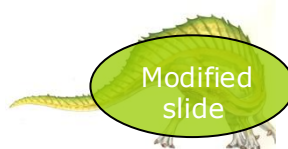
- If graph contains no cycles  $\Rightarrow$  no deadlock
- If graph contains a cycle  $\Rightarrow$ 
  - if only one instance per resource type, then deadlock
  - if several instances per resource type, **possibility** of deadlock





# Methods for Handling Deadlocks

- Ensure that the system will **never** enter a deadlock state:
  - **Deadlock prevention** (invalidate one of the 4 conditions for deadlock)
  - **Deadlock avoidance** – determine before granting a resource whether it may lead to deadlock e.g., Banker's algorithm
- Could also do things like...
  - Allow the system to enter a deadlock state and then recover
  - Ignore the problem and pretend that deadlocks never occur in the system.
    - ▶ **Often this is what happens... “it froze so we rebooted it”**



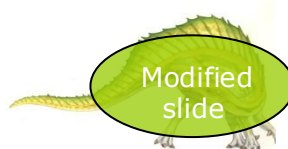


# Deadlock Prevention

---

Invalidate one of the four necessary conditions for deadlock:

- **Mutual Exclusion** – not required for sharable resources (e.g., read-only files); must hold for non-sharable resources
  - Some resources are intrinsically non-sharable!
  
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
  - Require process to request and be allocated all its resources before it begins execution /// allow process to request resources only when the process has none allocated to it.
  - Low resource utilization; starvation possible





# Deadlock Prevention (Cont.)

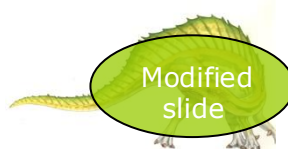
---

## ■ No Preemption:

- If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released **(try again later?)**
- Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting

## ■ Circular Wait:

- Impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration (responsibility of the programmer)

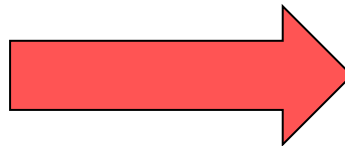




# Deadlock with Semaphores

- Data:
  - A semaphore `s1` initialized to 1
  - A semaphore `s2` initialized to 1
- Two processes `P1` and `P2`

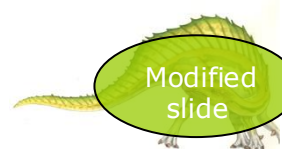
- `P1`:  
`wait(s1)`  
`wait(s2)`
- `P2`:  
`wait(s2)`  
`wait(s1)`



**No deadlock possible  
between `P1` and `P2`!**



- `P1`:  
`wait(s1)`  
`wait(s2)`
- `P2`:  
`wait(s1)`  
`wait(s2)`



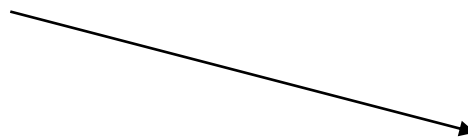


# Circular Wait

- Invalidating the circular wait condition is most common.
- Simply assign each resource (i.e., mutex locks) a unique number.
- Resources must be acquired in order.
- If:

```
first_mutex = 1
second_mutex = 5
```

code for `thread_two` could not be written as follows:

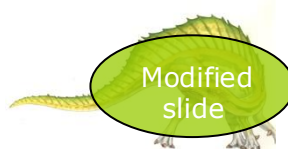


```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
 pthread_mutex_lock(&first_mutex);
 pthread_mutex_lock(&second_mutex);
 /**
 * Do some work
 */
 pthread_mutex_unlock(&second_mutex);
 pthread_mutex_unlock(&first_mutex);

 pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
 pthread_mutex_lock(&second_mutex);
 pthread_mutex_lock(&first_mutex);
 /**
 * Do some work
 */
 pthread_mutex_unlock(&first_mutex);
 pthread_mutex_unlock(&second_mutex);

 pthread_exit(0);
}
```





- 
- That's deadlock prevention
  - Let's look at deadlock avoidance



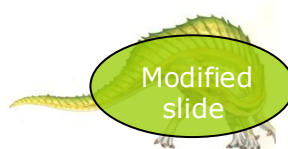


# Deadlock Avoidance

---

Requires that the system has some additional ***a priori*** information available

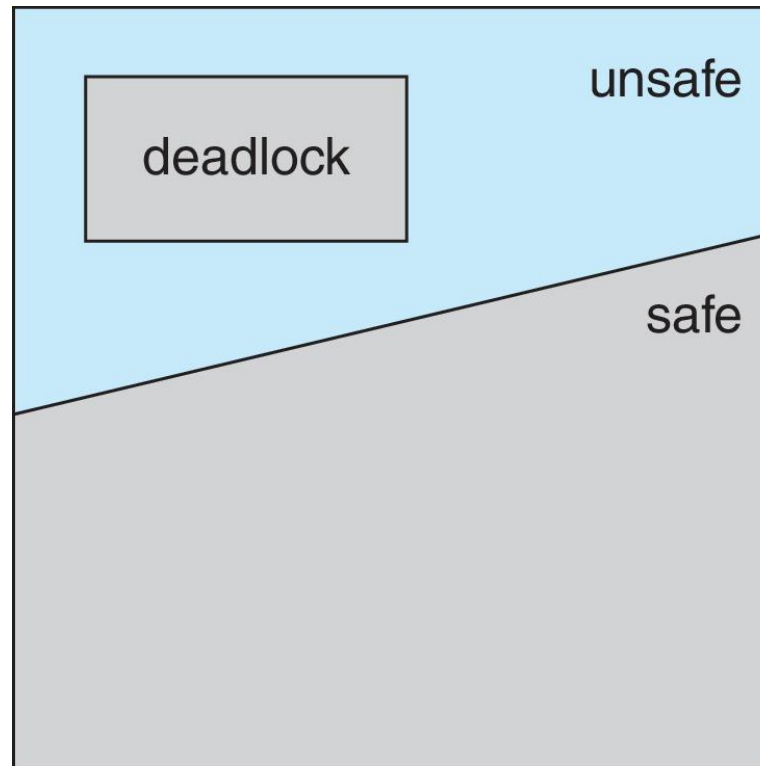
- Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
- The **deadlock-avoidance algorithm** dynamically examines the resource-allocation **state** to ensure that there can never be a circular-wait condition
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes





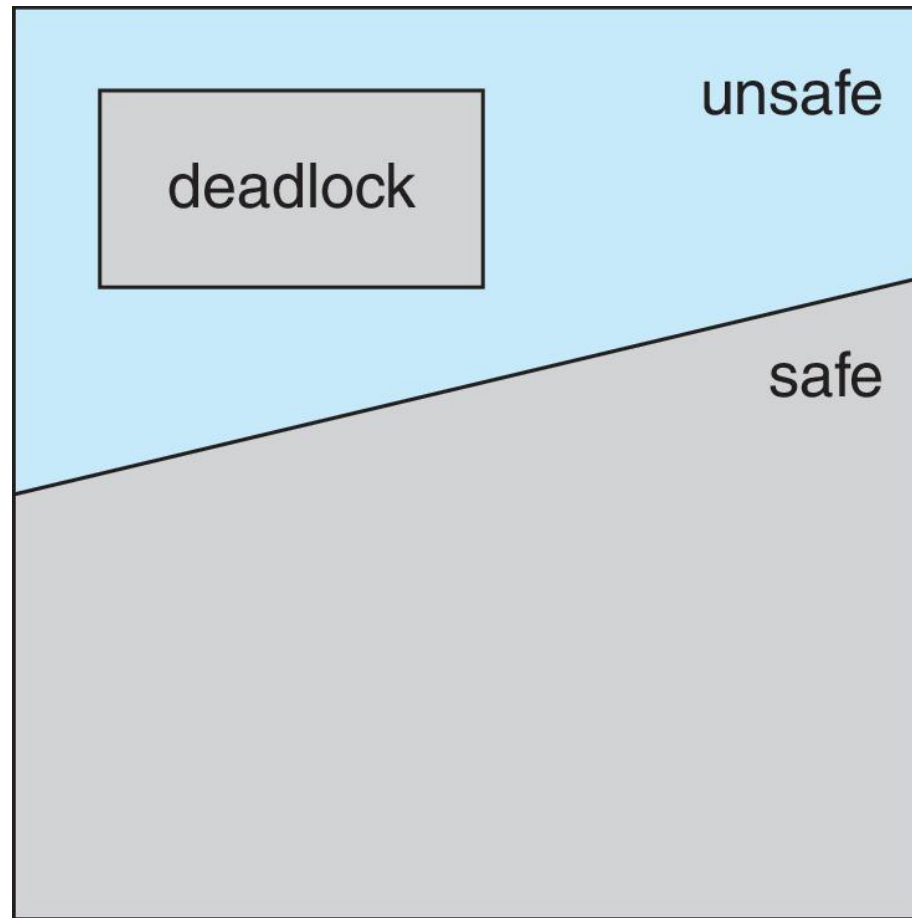
# Basic Facts

- If a system is in safe state  $\Rightarrow$  no deadlocks
- If a system is in unsafe state  $\Rightarrow$  possibility of deadlock
- Avoidance  $\Rightarrow$  ensure that a system will never enter an unsafe state.





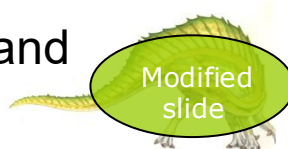
# Safe, Unsafe, Deadlock State





# Safe State

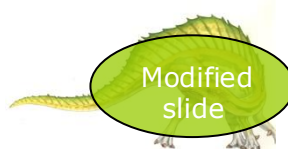
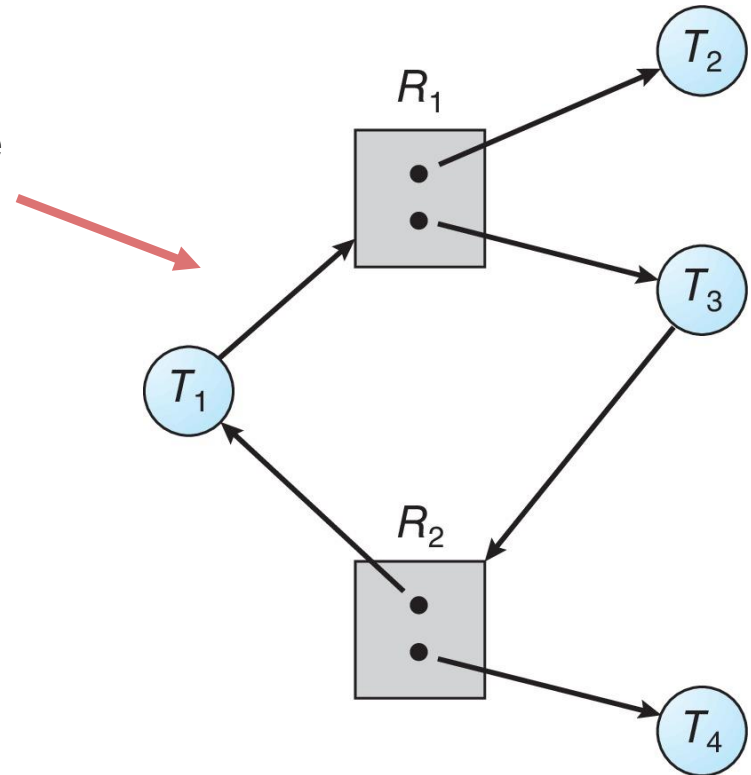
- When a process requests an available resource, system must decide if immediate allocation leaves the system in a **safe state**
- System is in **safe state** if there exists **a** sequence  $\langle P_1, P_2, \dots, P_n \rangle$  of ALL the processes in the system such that for each  $P_i$ , the resources that  $P_i$  **can still request** can be satisfied by currently available resources + resources held by all the  $P_j$ , with  $j < i$ 
  - e.g., if we know  $P_3$  might need  $X$  resources, and  $Y$  resources are already in use (by  $P_0, P_1, P_2$ ), are  $X+Y \leq \text{available}$ ?  
**i.e., is there a way/order that we can allocate the resources to let processes finish, so that there are resources for the next process**
- So:
  - If  $P_i$  resource needs are not immediately available, then  $P_i$  waits until all  $P_j$  have finished (resource becomes available)
  - When  $P_j$  is finished,  $P_i$  can obtain needed resources, execute, return allocated resources, and terminate
  - When  $P_i$  terminates,  $P_{i+1}$  can obtain its needed resources, and so on





# Avoidance Algorithms

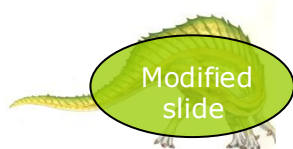
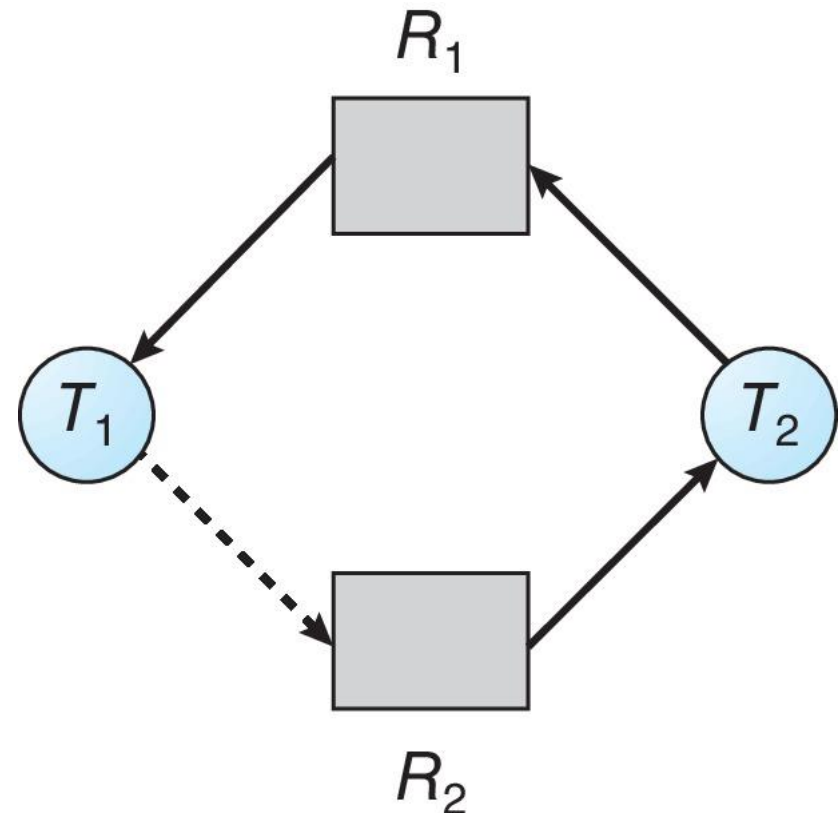
- Single instance of a resource type
  - Use a resource-allocation graph
- Multiple instances of a resource type
  - Use the Banker's Algorithm





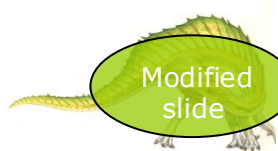
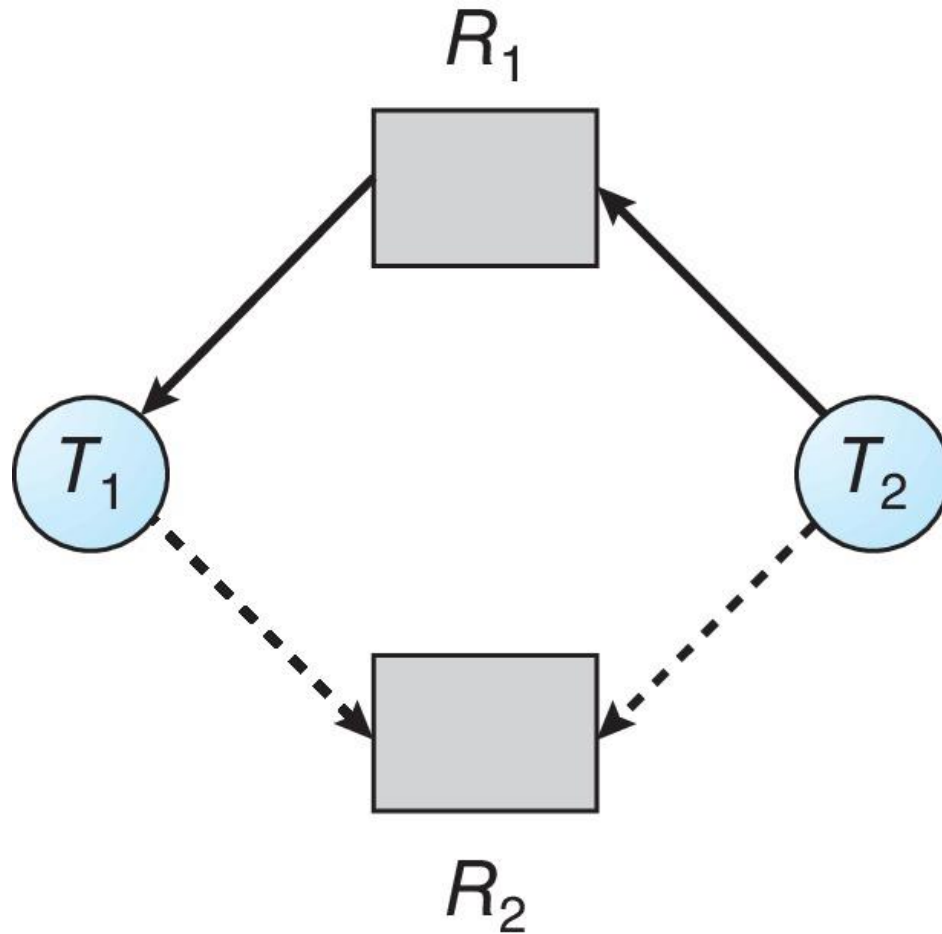
# Resource-Allocation Graph Scheme

- **Claim edge**  $P_i \rightarrow R_j$  indicated that process  $P_j$  **may** request resource  $R_j$ ; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources **must be claimed a priori** in the system



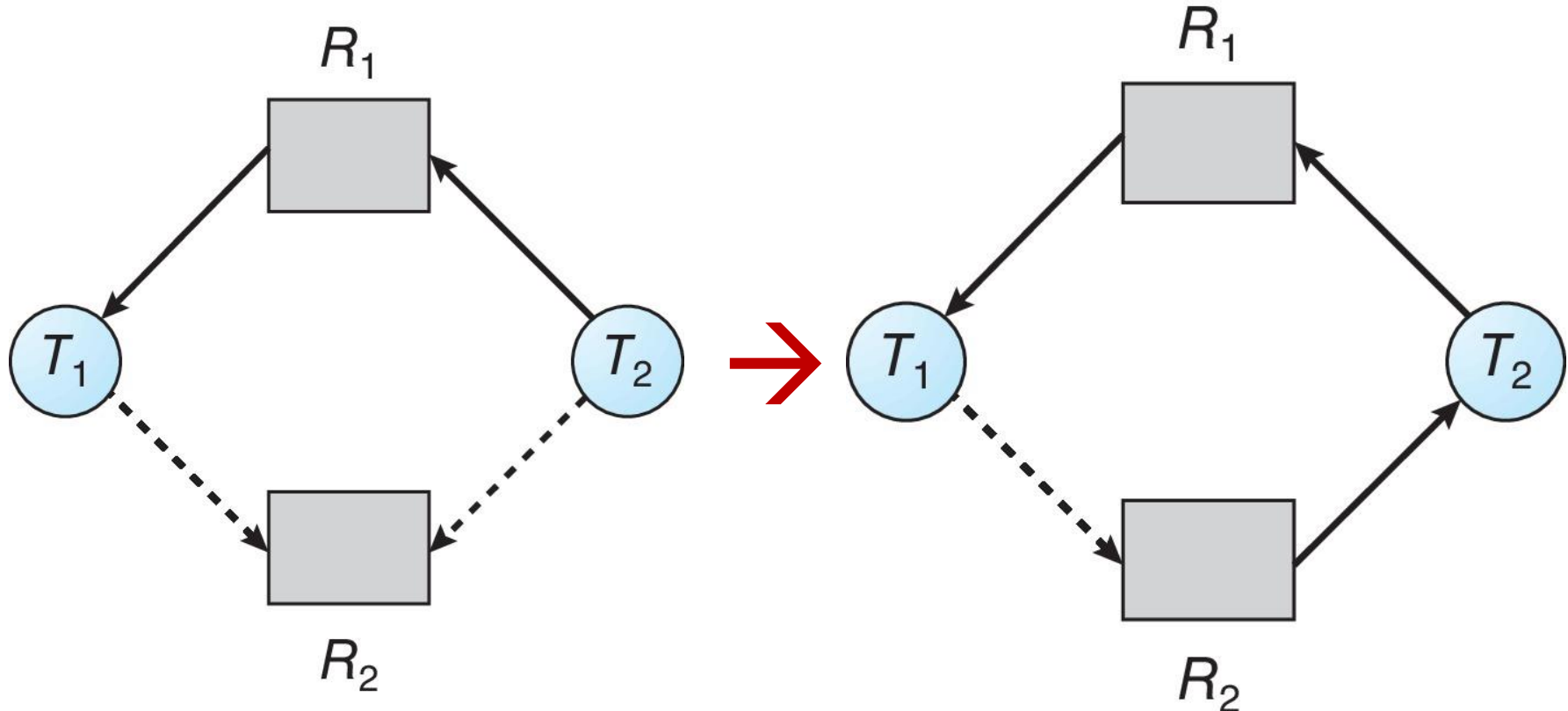


# Unsafe State In Resource-Allocation Graph

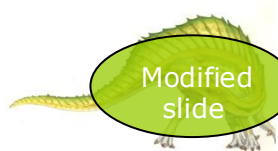




# Unsafe State In Resource-Allocation Graph

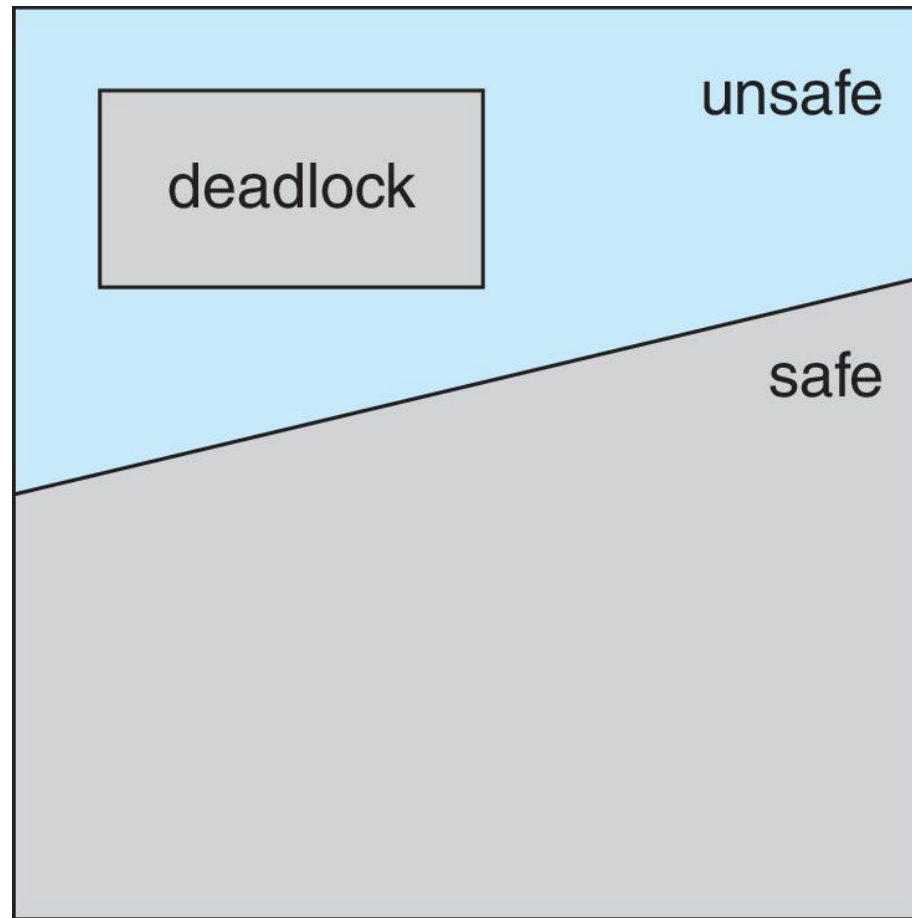


**Unsafe: Potential Deadlock!**





# Safe, Unsafe, Deadlock State





# Resource-Allocation Graph Algorithm

---

- Suppose that process  $P_i$  requests a resource  $R_j$
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph



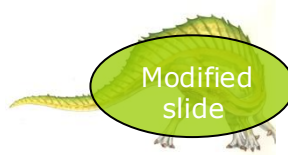




# Banker's Algorithm

---

- **Multiple** instances of resources
- Each process must a priori claim **maximum** use
- When a process requests a resource, it may have to wait
- When a process gets all its resources it must return them in a finite amount of time



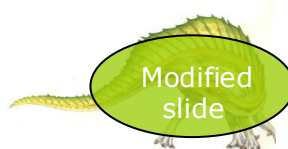


# Data Structures for the Banker's Algorithm

Let  $n$  = number of processes, and  $m$  = number of resources types.

- **Available:** Vector of length  $m$ . If  $available[j] = k$ , there are  $k$  instances of resource type  $R_j$  available
- **Max:**  $n \times m$  matrix. If  $Max[i,j] = k$ , then process  $P_i$  may request at most  $k$  instances of resource type  $R_j$
- **Allocation:**  $n \times m$  matrix. If  $Allocation[i,j] = k$  then  $P_i$  is currently allocated  $k$  instances of  $R_j$
- **Need:**  $n \times m$  matrix. If  $Need[i,j] = k$ , then  $P_i$  may need  $k$  more instances of  $R_j$  to complete its task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$





# Safety Algorithm

1. Let **Work** and **Finish** be vectors of length  $m$  and  $n$ , respectively.  
Initialize:

**Work** = Available (what's currently available)

**Finish** [ $i$ ] = false for  $i = 0, 1, \dots, n-1$

2. Find an  $i$  (process) such that both:

(a) **Finish** [ $i$ ] = false

(b) **Need** $_i \leq$  **Work**

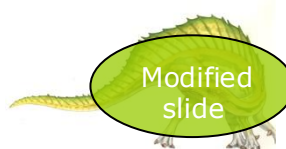
**Find a process  
that we've  
resources for**

If **no** such  $i$  exists, go to step 4

3. **Work** = **Work** + **Allocation** $_i$ ;  
**Finish** [ $i$ ] = true  
go to step 2

**Process is  
/ can  
finish(ed)!**

4. If **Finish** [ $i$ ] == true for all  $i$ , then the system is in a safe state





# Resource-Request Algorithm for Process $P_i$

**$Request_i$**  = request vector for process  $P_i$ . If  **$Request_i[j] = k$**  then process  $P_i$  wants  $k$  instances of resource type  $R_j$

1. If  **$Request_i \leq Need_i$**  go to step 2. Otherwise, raise error condition, since process has **exceeded** its **maximum claim**
2. If  **$Request_i \leq Available$** , go to step 3. Otherwise,  $P_i$  **must wait**, since resources are not available
3. **Pretend** to allocate requested resources to  $P_i$  by modifying the state as follows:

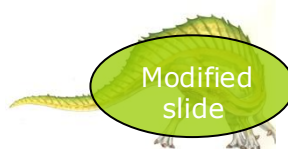
$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

## 4. We run our safety algorithm

- If safe  $\Rightarrow$  the resources are allocated to  $P_i$
- If unsafe  $\Rightarrow P_i$  must wait, and the old resource-allocation state is restored





# Example of Banker's Algorithm

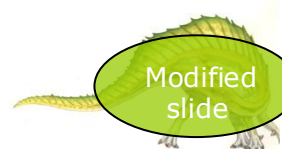
- 5 processes  $P_0$  through  $P_4$ ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

- Snapshot at time  $T_0$ :

|       | <u>Allocation</u> | <u>Max</u> | <u>Available</u> |
|-------|-------------------|------------|------------------|
|       | A B C             | A B C      | A B C            |
| $P_0$ | 0 1 0             | 7 5 3      | 3 3 2            |
| $P_1$ | 2 0 0             | 3 2 2      |                  |
| $P_2$ | 3 0 2             | 9 0 2      |                  |
| $P_3$ | 2 1 1             | 2 2 2      |                  |
| $P_4$ | 0 0 2             | 4 3 3      |                  |



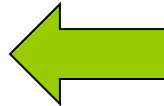


## Example (Cont.)

- The content of the matrix **Need** is defined to be **Max – Allocation**

(**may**) Need

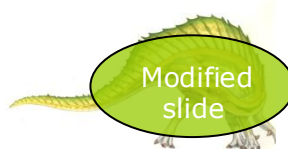
|       | A | B | C |
|-------|---|---|---|
| $P_0$ | 7 | 4 | 3 |
| $P_1$ | 1 | 2 | 2 |
| $P_2$ | 6 | 0 | 0 |
| $P_3$ | 0 | 1 | 1 |
| $P_4$ | 4 | 3 | 1 |



|       | <u>Allocation</u> |   |   | <u>Max</u> |   |   |
|-------|-------------------|---|---|------------|---|---|
|       | A                 | B | C | A          | B | C |
| $P_0$ | 0                 | 1 | 0 | 7          | 5 | 3 |
| $P_1$ | 2                 | 0 | 0 | 3          | 2 | 2 |
| $P_2$ | 3                 | 0 | 2 | 9          | 0 | 2 |
| $P_3$ | 2                 | 1 | 1 | 2          | 2 | 2 |
| $P_4$ | 0                 | 0 | 2 | 4          | 3 | 3 |

| <u>Available</u> |   |   |
|------------------|---|---|
| A                | B | C |
| 3                | 3 | 2 |

- Is the system in a safe state if we fully allocate for in sequence  $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ ?





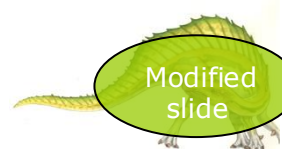
# Example: $P_1$ Request (1,0,2)

- Check that Request  $\leq$  Available, that is,  $(1,0,2) \leq (3,3,2) \Rightarrow \text{true}$

|       | <u>Allocation</u> | <u>Need</u> | <u>Available</u> |
|-------|-------------------|-------------|------------------|
|       | A B C             | A B C       | A B C            |
| $P_0$ | 0 1 0             | 7 4 3       | 2 3 0            |
| $P_1$ | 3 0 2             | 0 2 0       |                  |
| $P_2$ | 3 0 2             | 6 0 0       |                  |
| $P_3$ | 2 1 1             | 0 1 1       |                  |
| $P_4$ | 0 0 2             | 4 3 1       |                  |

|       | <u>Allocation</u> | <u>Max</u> | <u>Available</u> |
|-------|-------------------|------------|------------------|
|       | A B C             | A B C      | A B C            |
| $P_0$ | 0 1 0             | 7 5 3      | 3 3 2            |
| $P_1$ | 2 0 0             | 3 2 2      |                  |
| $P_2$ | 3 0 2             | 9 0 2      |                  |
| $P_3$ | 2 1 1             | 2 2 2      |                  |
| $P_4$ | 0 0 2             | 4 3 3      |                  |

- Executing safety algorithm shows that sequence  $\langle P_1, P_3, P_4, P_0, P_2 \rangle$  satisfies safety requirement
- Can request for (3,3,0) by  $P_4$  be granted?
- Can request for (0,2,0) by  $P_0$  be granted? ← ?





# Example of Banker's Algorithm

- 5 processes  $P_0$  through  $P_4$ ;

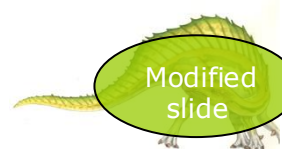
3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

- Can request for (0,2,0) by  $P_0$  be granted? (let's play it out)

- Snapshot at time  $T_0$ :

|       | <u>Allocation</u> | <u>Max</u> | <u>Available</u> | ( <b>may</b> ) <u>Need</u> |
|-------|-------------------|------------|------------------|----------------------------|
|       | A B C             | A B C      | A B C            | A B C                      |
| $P_0$ | 0 <b>3</b> 0      | 7 5 3      | 3 <b>1</b> 2     | $P_0$ 7 2 3                |
| $P_1$ | 2 0 0             | 3 2 2      |                  | $P_1$ 1 2 2                |
| $P_2$ | 3 0 2             | 9 0 2      |                  | $P_2$ 6 0 2                |
| $P_3$ | 2 1 1             | 2 2 2      |                  | $P_3$ 0 1 1                |
| $P_4$ | 0 0 2             | 4 3 3      |                  | $P_4$ 4 3 1                |





# Example of Banker's Algorithm

- 5 processes  $P_0$  through  $P_4$ ;

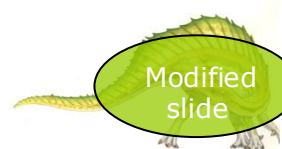
3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

- Can request for (0,3,0) by  $P_0$  be granted? (let's play it out)

- Snapshot at time  $T_0$ :

|       | <u>Allocation</u> | <u>Max</u> | <u>Available</u> | ( <b>may</b> ) <u>Need</u> |
|-------|-------------------|------------|------------------|----------------------------|
|       | A B C             | A B C      | A B C            | A B C                      |
| $P_0$ | 0 <b>4</b> 0      | 7 5 3      | 3 <b>0</b> 2     | $P_0$ 7 1 3                |
| $P_1$ | 2 0 0             | 3 2 2      |                  | $P_1$ 1 2 2                |
| $P_2$ | 3 0 2             | 9 0 2      |                  | $P_2$ 6 0 2                |
| $P_3$ | 2 1 1             | 2 2 2      |                  | $P_3$ 0 1 1                |
| $P_4$ | 0 0 2             | 4 3 3      |                  | $P_4$ 4 3 1                |



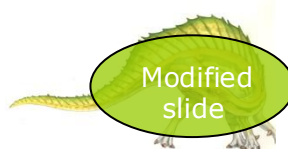




# Deadlock Detection

---

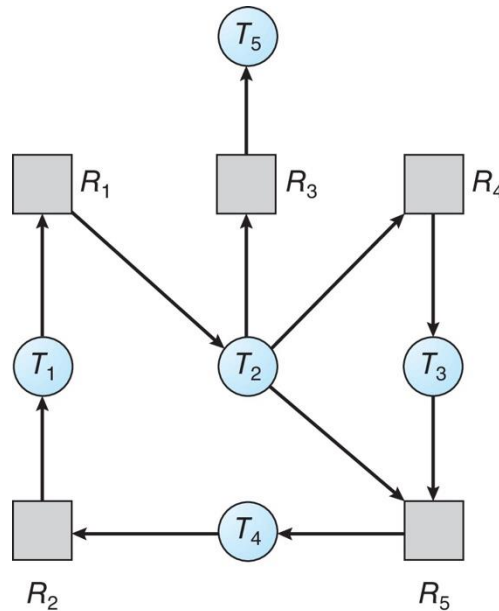
- We looked at prevention and avoidance ... But perhaps we didn't prevent or avoid deadlock, and just let it happen....
- Allow system to enter deadlock state
- Use a detection algorithm to know when this happens, and then ...
- Recovery scheme





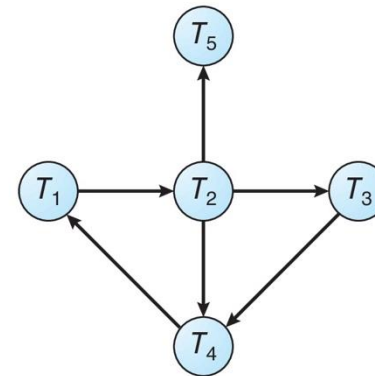
# Single Instance of Each Resource Type

- Maintain **wait-for** graph
  - Nodes are processes
  - $P_i \rightarrow P_j$  if  $P_i$  is waiting for  $P_j$
- Periodically invoke an algorithm that searches for a **cycle** in the graph.



(a)

Resource-Allocation Graph



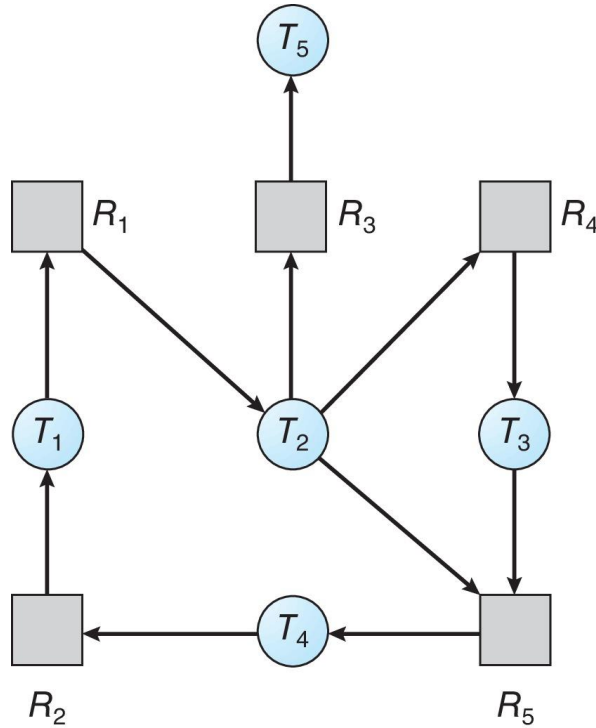
(b)

Corresponding wait-for graph

Modified slide

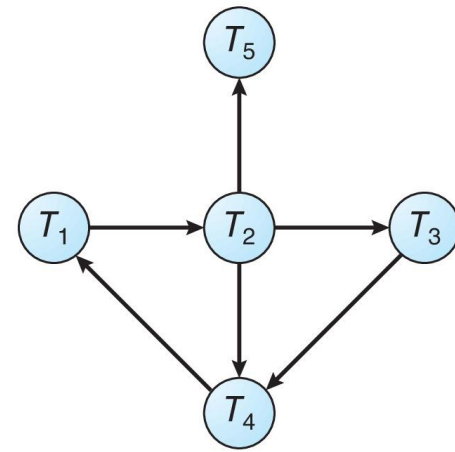


# Resource-Allocation Graph and Wait-for Graph



(a)

Resource-Allocation Graph



(b)

Corresponding **wait-for** graph





# Several Instances of a Resource Type

---

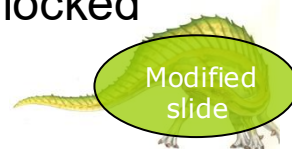
- **Available:** A vector of length  $m$  indicates the number of available resources of each type
- **Allocation:** An  $n \times m$  matrix defines the number of resources of each type currently allocated to each process
- **Request:** An  $n \times m$  matrix indicates the current request of each process. If **Request**  $[i][j] = k$ , then process  $P_i$  is requesting  $k$  more instances of resource type  $R_j$ .





# Detection Algorithm

1. Let **Work** and **Finish** be vectors of length **m** and **n**, respectively  
Initialize:
  - a) **Work = Available**
  - b) For  $i = 1, 2, \dots, n$ , if  $\text{Allocation}_i \neq 0$ , then **Finish[i] = false**; otherwise, **Finish[i] = true**
2. Find an index **i** such that both:
  - a) **Finish[i] == false**
  - b) **Request<sub>i</sub> ≤ Work**If no such **i** exists, go to step 4
3. **Work = Work + Allocation<sub>i</sub>**  
**Finish[i] = true**  
go to step 2
4. If **Finish[i] == false**, for some **i**,  $1 \leq i \leq n$ , then the system is in deadlock state. Moreover, if **Finish[i] == false**, then **P<sub>i</sub>** is deadlocked





# Example of Detection Algorithm

- Five processes  $P_0$  through  $P_4$ ; three resource types A (7 instances), B (2 instances), and C (6 instances)
- Snapshot at time  $T_0$ :

|       | <u>Allocation</u> | <u>Request</u> | <u>Available</u> |
|-------|-------------------|----------------|------------------|
|       | A B C             | A B C          | A B C            |
| $P_0$ | 0 1 0             | 0 0 0          | 0 0 0            |
| $P_1$ | 2 0 0             | 2 0 2          |                  |
| $P_2$ | 3 0 3             | 0 0 0          |                  |
| $P_3$ | 2 1 1             | 1 0 0          |                  |
| $P_4$ | 0 0 2             | 0 0 2          |                  |

- Sequence  $\langle P_0, P_2, P_3, P_1, P_4 \rangle$  will result in ***Finish[i] = true*** for all  $i$



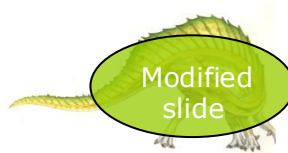


# Example of Detection Algorithm

$P_2$  requests an additional instance of type **C**

|       | <u>Allocation</u> | <u>Request</u> | <u>Available</u> |
|-------|-------------------|----------------|------------------|
|       | <i>A B C</i>      | <i>A B C</i>   | <i>A B C</i>     |
| $P_0$ | 0 1 0             | 0 0 0          | 0 0 0            |
| $P_1$ | 2 0 0             | 2 0 2          |                  |
| $P_2$ | 3 0 3             | 0 0 <b>1</b>   |                  |
| $P_3$ | 2 1 1             | 1 0 0          |                  |
| $P_4$ | 0 0 2             | 0 0 2          |                  |

**But what if?**





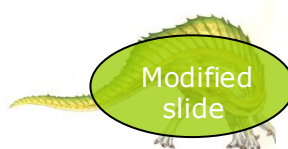
# Example (Cont.) but what if?

- $P_2$  requests an additional instance of type **C**

|       | <u>Request</u> |   |          |
|-------|----------------|---|----------|
|       | A              | B | C        |
| $P_0$ | 0              | 0 | 0        |
| $P_1$ | 2              | 0 | 2        |
| $P_2$ | 0              | 0 | <b>1</b> |
| $P_3$ | 1              | 0 | 0        |
| $P_4$ | 0              | 0 | 2        |

**But what if?**

- State of system?
  - Can reclaim resources held by process  $P_0$ , but insufficient resources to fulfill other processes; requests
  - Deadlock exists, consisting of processes  $P_1$ ,  $P_2$ ,  $P_3$ , and  $P_4$



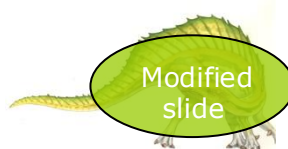


# Recovery from Deadlock: **Process Termination**

---

- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?
  1. Priority of the process
  2. How long process has computed, and how much longer to completion
  3. Resources the process has used
  4. Resources process needs to complete
  5. How many processes will need to be terminated
  6. Is process interactive or batch?
  7. ...

**How to pick  
from bad  
options?**





# Recovery from Deadlock: Resource Preemption

---

- **Selecting a victim** – minimize cost
- **Rollback** – return to some safe state, restart process for that state (not always possible though, programs affect real world)
- **Starvation** – same process may always be picked as victim, include number of rollback in cost factor
- Messy ...

